

PRACTICALWEEK6

TASK1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
jhansi_priya@Jhansi-Priya: ~/ X + v
jhansi_priya@Jhansi-Priya:~$ mkdir Practical6
jhansi_priya@Jhansi-Priya:~$ cd Practical6
jhansi_priya@Jhansi-Priya:~/Practical6$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Practical6$ nano client.c
jhansi_priya@Jhansi-Priya:~/Practical6$ gcc task1.c -o task1
```

```
jhansi_priya@Jhansi-Priya:~/Practical6$ gcc client.c -o client
jhansi_priya@Jhansi-Priya:~/Practical6$ ls
client client.c task1 task1.c
jhansi_priya@Jhansi-Priya:~/Practical6$ ./client
Client PID: 542
Enter message: Hello Server
open server FIFO: No such file or directory
jhansi_priya@Jhansi-Priya:~/Practical6$ |
```

```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>

#define SERVER_FIFO "/tmp/my_server_fifo"
#define MAX_MSG 256

struct request {
    pid_t pid;
    char message[MAX_MSG];
};

int main()
{
    int fd;
    struct request req;

    // Remove old FIFO if it exists
    unlink(SERVER_FIFO);

    // Create named pipe
    if (mkfifo(SERVER_FIFO, 0666) == -1)
    {
        perror("mkfifo");
        exit(1);
    }

    printf("Server started...\n");
    printf("Server FIFO: %s\n", SERVER_FIFO);
    printf("Waiting for clients...\n");

    // Open FIFO for reading and writing
    // O_RDWR prevents EOF when no client is connected
    fd = open(SERVER_FIFO, O_RDWR);

    if (fd == -1)
    {
        perror("open");
        exit(1);
    }

    while (1)
    {
        ssize_t n = read(fd, &req, sizeof(req));
        if (n == sizeof(req))
        {
            printf("\nReceived message from Client PID %d: %s\n",
                req.pid, req.message);

            // Create child process for each client request
            pid_t child = fork();
            if (child == 0)
            {
                char response_fifo[100];
```

```
GNU nano 7.2 task1.c *
if (n == sizeof(req))
{
    printf("\nReceived message from Client PID %d: %s\n",
        req.pid, req.message);

    // Create child process for each client request
    pid_t child = fork();
    if (child == 0)
    {
        char response_fifo[100];
        sprintf(response_fifo,
            "/tmp/response_%d",
            req.pid);

        // Process the message
        char response[MAX_MSG];
        snprintf(response,
            MAX_MSG,
            "Server processed your message: %s",
            req.message);

        // Open client's response FIFO
        int response_fd =
            open(response_fifo, O_WRONLY);

        if (response_fd == -1)
        {
            perror("open response FIFO");
            exit(1);
        }

        write(response_fd,
            response,
            strlen(response) + 1);

        close(response_fd);
        printf("Response sent to Client PID %d\n",
            req.pid);

        exit(0);
    }
    else if (child < 0)
    {
        perror("fork");
    }
}

// Clean up finished child processes
while (waitpid(-1, NULL, WNOHANG) > 0);

close(fd);
unlink(SERVER_FIFO);

return 0;
}
```

```
GNU nano 7.2 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>

#define SERVER_FIFO "/tmp/my_server_fifo"
#define MAX_MSG 256

struct request {
    pid_t pid;
    char message[MAX_MSG];
};

int main()
{
    int server_fd, response_fd;
    struct request req;

    char response_fifo[100];
    char response[MAX_MSG];

    // Get client's process ID
    req.pid = getpid();

    // Create unique response FIFO
    sprintf(response_fifo,
        "/tmp/response_%d",
        req.pid);

    unlink(response_fifo);

    if (mkfifo(response_fifo, 0666) == -1)
    {
        perror("mkfifo");
        exit(1);
    }

    printf("Client PID: %d\n", req.pid);
    printf("Enter message: ");
    fgets(req.message, MAX_MSG, stdin);

    // Remove newline
    req.message[strcspn(req.message, "\n")] = '\0';

    // Open server FIFO
    server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1)
    {

```

```
GNU nano 7.2 client.c *
    perror("mkfifo");
    exit(1);
}

printf("Client PID: %d\n", req.pid);
printf("Enter message: ");
fgets(req.message, MAX_MSG, stdin);

// Remove newline
req.message[strcspn(req.message, "\n")] = '\0';

// Open server FIFO
server_fd = open(SERVER_FIFO, O_WRONLY);
if (server_fd == -1)
{
    perror("open server FIFO");
    unlink(response_fifo);
    exit(1);
}

// Send request
write(server_fd, &req, sizeof(req));
close(server_fd);

printf("Message sent to server.\n");
printf("Waiting for server response...\n");

// Open response FIFO
response_fd = open(response_fifo, O_RDONLY);
if (response_fd == -1)
{
    perror("open response FIFO");
    unlink(response_fifo);
    exit(1);
}

// Read response
read(response_fd, response, MAX_MSG);
printf("Server response: %s\n", response);
close(response_fd);

// Remove client's FIFO
unlink(response_fifo);

return 0;
}
```

TASK2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
GNU nano 7.2 signal_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

void handle_sigint(int sig)
{
    const char msg[] =
        "\nSIGINT received! Program will exit.\n";

    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    exit(0);
}

void handle_sigterm(int sig)
{
    const char msg[] =
        "\nSIGTERM received! Program will terminate.\n";

    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    exit(0);
}

void handle_sigusr1(int sig)
{
    const char msg[] =
        "\nSIGUSR1 received! User-defined event occurred.\n";

    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}

int main()
{
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ▼
GNU nano 7.2 signal_demo.c *

int main()
{
    struct sigaction sa_int;
    struct sigaction sa_term;
    struct sigaction sa_usr1;

    // Clear structures
    memset(&sa_int, 0, sizeof(sa_int));
    memset(&sa_term, 0, sizeof(sa_term));
    memset(&sa_usr1, 0, sizeof(sa_usr1));

    // Assign handlers
    sa_int.sa_handler = handle_sigint;
    sa_term.sa_handler = handle_sigterm;
    sa_usr1.sa_handler = handle_sigusr1;

    // Register signal handlers
    sigaction(SIGINT, &sa_int, NULL);
    sigaction(SIGTERM, &sa_term, NULL);
    sigaction(SIGUSR1, &sa_usr1, NULL);

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());

    printf("\nWaiting for signals...\n");
    printf("Press Ctrl+C for SIGINT.\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ▼
jhansi_priya@Jhansi-Priya:~$ mkdir signal_task
jhansi_priya@Jhansi-Priya:~$ cd signal_task
jhansi_priya@Jhansi-Priya:~/signal_task$ nano signal_demo.c
jhansi_priya@Jhansi-Priya:~/signal_task$ gcc signal_demo.c -o signal_demo
jhansi_priya@Jhansi-Priya:~/signal_task$ ls
signal_demo  signal_demo.c
jhansi_priya@Jhansi-Priya:~/signal_task$ ./signal_demo
Signal handling program started.
Process ID (PID): 608

Waiting for signals...
Press Ctrl+C for SIGINT.

SIGUSR1 received! User-defined event occurred.

SIGUSR1 received! User-defined event occurred.

SIGUSR1 received! User-defined event occurred.

SIGUSR1 received! User-defined event occurred.

SIGINT received! Program will exit.
jhansi_priya@Jhansi-Priya:~/signal_task$ ./signal_demo
Signal handling program started.
Process ID (PID): 658

Waiting for signals...
Press Ctrl+C for SIGINT.

SIGTERM received! Program will terminate.
jhansi_priya@Jhansi-Priya:~/signal_task$ |
```

jhansi_priya@Jhansi-Priya: ~

```
jhansi_priya@Jhansi-Priya:~$ kill -USR1 608
jhansi_priya@Jhansi-Priya:~$ Kill -USR1 608
Command 'Kill' not found, did you mean:
  command 'mill' from snap mill (6.7.1)
  command 'kill' from deb procs (2:4.0.4-4ubuntu3.2)
See 'snap info <snapname>' for additional versions.
jhansi_priya@Jhansi-Priya:~$ kill -USR1 608
jhansi_priya@Jhansi-Priya:~$ kill -USR1 608
jhansi_priya@Jhansi-Priya:~$ kill -USR1 608
jhansi_priya@Jhansi-Priya:~$ kill -INT 608
jhansi_priya@Jhansi-Priya:~$ kill -TERM 658
jhansi_priya@Jhansi-Priya:~$ |
```

